

CSE222 HOMEWORK #2

Q1)

$$\begin{aligned}
 a) \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} &\Rightarrow \lim_{n \rightarrow \infty} \frac{(n^2 - 3n)^2}{(5n^3 + n)} = \lim_{n \rightarrow \infty} \frac{\cancel{n^2} (n-3)^2}{\cancel{n} (5n^2 + 1)} \xrightarrow{\text{dividing both numerator and denominator by "n"}} \lim_{n \rightarrow \infty} \frac{(n-3)^2}{5n + \cancel{1}_0} \\
 &= \lim_{n \rightarrow \infty} \frac{n^2 - 6n + 9}{5n} = \lim_{n \rightarrow \infty} \frac{n - 6 + \cancel{\frac{9}{n}}_0}{5} = \lim_{n \rightarrow \infty} \frac{n-6}{5} = \infty
 \end{aligned}$$

dividing by "n"

Therefore $f(n) = \Omega(g(n))$

$$\begin{aligned}
 b) \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} &\Rightarrow \lim_{n \rightarrow \infty} \frac{n^3}{\log_2 n^4} \xrightarrow{\text{L'Hopital}} \lim_{n \rightarrow \infty} \frac{n^3}{4 \log_2 n} = \lim_{n \rightarrow \infty} \frac{3n^2}{\frac{4}{n \ln 2}} \\
 &= \lim_{n \rightarrow \infty} \frac{3n^3 \cdot \ln 2}{4} = \frac{3}{4} \cdot \ln 2 \cdot \lim_{n \rightarrow \infty} \underbrace{n^3}_{\infty} = \frac{3}{4} \ln 2 \cdot \infty = \infty
 \end{aligned}$$

Therefore $f(n) = \Omega(g(n))$

$$\begin{aligned}
 c) \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} &\Rightarrow \lim_{n \rightarrow \infty} \frac{5n \cdot \log_2(4n)}{n \cdot \log_2(5^n)} = 5 \cdot \lim_{n \rightarrow \infty} \frac{\log_2(4n)}{\log_2(5^n)} \xrightarrow{\text{L'Hopital}} 5 \cdot \lim_{n \rightarrow \infty} \frac{\frac{1}{4n \ln 2}}{\frac{1}{5^n \ln 2}} \\
 &= 5 \cdot \lim_{n \rightarrow \infty} \frac{5^n}{(5^n)' \cdot n} = 5 \cdot \lim_{n \rightarrow \infty} \frac{\cancel{5^n}}{\underbrace{5^n \ln 5 \cdot n}_{\text{derivative of } 5^n}} = 5 \cdot \lim_{n \rightarrow \infty} \frac{1}{\ln 5 \cdot n} = \frac{5}{\ln 5} \lim_{n \rightarrow \infty} \frac{1}{n} = 0
 \end{aligned}$$

Therefore $f(n) = O(g(n))$

$$d) \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \Rightarrow \lim_{n \rightarrow \infty} \frac{n^n}{10^n} = \lim_{n \rightarrow \infty} \left(\frac{n}{10}\right)^n = \lim_{n \rightarrow \infty} \underbrace{e^{\ln\left(\left(\frac{n}{10}\right)^n\right)}}_{\substack{\text{from the rule} \\ u = e^{\ln(u)}}}$$

$$= \lim_{n \rightarrow \infty} e^{n \cdot \ln\left(\frac{n}{10}\right)} \xrightarrow{\text{1st solution}} \lim_{n \rightarrow \infty} e^{\infty \cdot \ln\left(\frac{\infty}{10}\right)} = e^{\infty \cdot \ln(\infty)} = e^{\infty} = \infty$$

$$\begin{array}{l} \text{2nd} \\ \text{solution} \end{array} \rightarrow \underbrace{e^{\lim_{n \rightarrow \infty} n \ln\left(\frac{n}{10}\right)}}_{\substack{\text{moving the limit} \\ \text{under the exponential}}} = \lim_{n \rightarrow \infty} n \ln\left(\frac{n}{10}\right) = \infty$$

★ Therefore $f(n) = \Omega(g(n))$

$$e) \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \Rightarrow \lim_{n \rightarrow \infty} \frac{8n \cdot \sqrt[5]{2n}}{n \cdot \sqrt[3]{n}} = 8 \cdot \lim_{n \rightarrow \infty} \frac{2n^{1/5}}{n^{1/3}} = 8 \cdot 2^{1/5} \lim_{n \rightarrow \infty} \frac{n^{1/5}}{n^{1/3}}$$

$$= 8 \cdot 2^{1/5} \cdot \lim_{n \rightarrow \infty} n^{1/5 - 1/3} = 8 \cdot 2^{1/5} \cdot \lim_{n \rightarrow \infty} n^{-2/15} = 8 \cdot 2^{1/5} \cdot \lim_{n \rightarrow \infty} \frac{1}{n^{2/15}} = 8 \cdot 2^{1/5} \cdot \frac{1}{\infty} = \frac{8 \cdot 2^{1/5}}{\infty} = 0$$

Therefore $f(n) = O(g(n))$

Q2)

a) method A includes a for loop that iterates through the str-array.

Since this for loop continues till the str-array.length which is "n", worst time complexity of method A is $O(n)$.

b) method B includes 2 separate for loops and both of these for loops iterate through the length of the array. However, first for loop calls method A for every index of the loop which has a time complexity of $O(n)$. Since the for loop has a time complexity of "n" and in every iteration method A which has a "n" time complexity is called the time complexity of the first for loop is n times n which is n^2 . Time complexity of the second for loop is n too since it iterates through the length of the array. However, since n^2 is greater than n we omit it and take the time complexity as $O(n^2)$ instead of $n^2 + n$. Shortly the time complexity of method B is $O(n^2)$.

c) method C includes a nested for loop and both of these for loops iterate through the length of the array. In every iteration method B is called inside inner loop of the nested loop. Since the time complexity of nested loop is the square of the length of the array which is n^2 , and the time complexity of method B is n^2 as explained in option B. Since in every iteration method B which has a time complexity of n^2 is called and the nested loop has a time complexity of n^2 the time complexity of method C is n^2 times n^2 results in n^4 . Therefore, the time complexity of method C is $O(n^4)$.

d) method D includes a for loop iterates through the length of the array and inside this for loop there is an expression which is `str_array[i--] = " " ;` in this expression `i` is decremented and therefore in the first iteration `i` has become `-1`. Since `-1` is less than the length of the array the `"i < str_array.length"` condition is not satisfied because of that the loop continues. In the next iteration `i` is incremented in for loop statement `"i++"` and become `"0"` again then after `"i--"` it again becomes `"-1"`. Therefore the loop continues till infinity with `"i"` becoming `0` and `-1` consecutively being less than the length of the array so that the loop does not terminate. For this reason time complexity can not be calculated since the loop goes to infinity.

e) method E includes a for loop iterates through the length of the array inside the loop there is an if statement that checks if the current index of the array is blank or not and according to that if it is loop terminates. Since we are looking for the worst case I assumed that the every element of the array is filled with something. Therefore the for loop does not terminate since every element of the array is full. This results in the for loop to iterate through the length of the array without being terminated therefore it has a time complexity of the length of the array which is $O(n)$.

Q3)

a) Pseudo-code

$$\text{max_difference} = A[n-1] - A[0]$$

Explanation: The array A is sorted in ascending order which means that the smallest element in the array is the first element which is index zero and the biggest element in the array is the last element which is index $n-1$. Therefore the max difference is $A[n-1] - A[0]$ and the time complexity is $O(1)$ since there is only one operation.

b) Pseudo-code

$$\text{min} = A[0]$$

$$\text{max} = A[0]$$

FOR each element in A

IF $A[\text{element}]$ is greater than max // $A[i] > \text{max}$

$$\text{max} = A[\text{element}]$$

ENDIF

IF $A[\text{element}]$ is less than min // $A[i] < \text{min}$

$$\text{min} = A[\text{element}]$$

ENDIF

ENDFOR

$$\text{max_difference} = \text{max} - \text{min}$$

Explanation: In the pseudo-code I first assigned the min and max to the first index of the array A temporarily. Then I have written a for loop that iterates through the length of the array. In the loop for every index I checked if the current element is greater than our current max value or less than our current min value. Accordingly if it is I changed the current min or max value by assigning them to the current element of the index. Finally after the loop is finished and min and max values has been found by checking every element in the array A, I calculated the max-difference by subtracting the min value from max. Time complexity of this pseudo-code is $O(n)$ since there is a for loop iterates through our array A.

Muhammet Furkan Yilgiz

220104004364

M. Furkan